

Casting and Type Conversion

📝 Notes by Menna Reda

الاول كد اي حاجه في ال NET framework, عبارة عن Class

اي شئى عندي في ال NET. ال root تبعة ال object

Implicit Conversion:

بيتم تلقائيا ال compiler بيعمله لوحدة لان التحويل دا 100% امن يعني اي يعني ال compiler ضامن ان مش هيجصل اي data .loss

زي التحويل بين ال datatype اللي بتخزن ارقام بس بشروط

مثلا اقدر احوّل ال int ل double لآكن العكس مينفعش

احول char ل int لان ال char ليه قيمة رقم في ال ASCII

احول من datatype حجمها صغير ل datatype حجمها كبير

```
int myInt = 9;
double myDouble = myInt; // تحويل تلقائي من int إلى double

Console.WriteLine(myInt); // 9
Console.WriteLine(myDouble); // 9.0
```

ترتيب الحجم (من الأصغر للأكبر):

char -> int -> long -> float -> double

طبعا مينفعش اعمل العكس من الكبير للصغير هنا هيعمل Compile time error وهنا لازم اجبره بنفسه انه يعمل التحويل دا
ودا هو ال Explicit Conversion

Explicit Conversion:

ده بيحصل لما تحب تحول من نوع حجمه كبير لنوع حجمه أصغر، أو من نوع أعلى دقة لنوع أقل دقة. هنا ال Compiler بيرفض يعملها أوتوماتيك لأن فيه احتمالية لفقدان البيانات (Data loss)، ولأزم إنت ك مبرمج تقول: "أنا عارف أنا بعمل إيه ومتحمل المسؤولية".

- الطريقة: بنكتب اسم النوع الجديد بين أقواس () قبل المتغير المراد تحويله.

مثال:

```
double myDouble = 9.78;
int myInt = (int)myDouble; // Casting
long num = 12;
int n = (int)num;

Console.WriteLine(myDouble); // 9.78
Console.WriteLine(myInt); // 9
```

هنا الكومبايلر مش بيهتم بق هل ممكن يحصل lose لبعض الداتا او لا

Boxing Vs Unboxing

Boxing:

التحويل من داتا تايب value الي داتا تايب reference

Implicit Conversion دا من انواع ال

```
int x = 5;
object obj;
obj = x;
```

Unboxing

التحويل من داتا تايب reference الي داتا تايب value

Explicit Conversion دا من انواع ال

```
int x = 5;
object obj=x;
int y= (int)obj;
```



Non-Compatible Types

التحويل بين الأنواع غير المتوافقة (Non-Compatible Types)

لو حاولت تعمل Casting عادي بين نوعين مفيش بينهم علاقة (زي string و int)، الكود هيطلع Error ومش هيشغل.

فعشان احول من string ل int او من string ل numbers عموما بستخدم type.Parse()

```
string stringValue = "123";
int number = int.Parse(stringValue);
Console.WriteLine(number);

string ageText = "25";
int age = int.Parse(ageText);
```

ال type.Parse()

Parse() : ارقام ("abc" مثلاً) سريعة ومباشرة، بس لو النص جواه حروف مش أرقام :

او ارقام كبيرة اكبر من ان ال int او long ممكن تخزنه البرنامج هيقفل وهيطلع خطأ (Exception).

```
string stringValue = "12355555555555555555555555555555";
int number = int.Parse(stringValue);
Console.WriteLine(number);

//OverflowException: 'Value was either too large or too small for an Int32.'
```

عشان كد الافضل نستخدم ال TryParse()

TryParse() :
دي الطريقة الآمنة والمحترفة. بتحاول تحول النص، لو نجحت بترجع **true** وتحط القيمة في متغير، ولو فشلت بترجع **false** من غير ما البرنامج يقفل.

ال **TryParse(invalidData, out result)** بترجع **bool** لو قدر انه يحول ويخزن القيمة داخل المتغير اللي جواها **result**

```
string invalidData = "123X";
int result;

// out result يعني لو نجح حط النتيجة في متغير result
bool isSuccess = int.TryParse(invalidData, out result);

if (isSuccess) {
    Console.WriteLine($"{result}");
} else {
    Console.WriteLine("cannot conver this value");
}

string stringValue = "123555";
//int number;
if (int.TryParse(stringValue, out int number))
{
    Console.WriteLine(number);
}
else Console.WriteLine("cannot converting this value");
```

وكدا اكون حميت البرنامج من ال **Exception**.

Convert class

دي فئة جاهزة في **#C** تحتوي على ميثودز (Methods) لتحويل أي نوع لنوع ثاني.

```
string textValue = "45";
int num = Convert.ToInt32(textValue); // Convert string to int

double d = 5.5;
string textdouble = Convert.ToString(d); // Convert double to string
```

ملحوظة: هنا بردو لو جيت احول من **string** ل **int** و ال **string** يحتوي ع احرف او قيمه تتعدي **size** ال **int** هنا بردو هيضرب و هيطلع **exception**

ملخص:

وجه المقارنة	Implicit Conversion	Explicit Conversion (Casting)	Convert / Parse
التحويل بيتم تلقائياً بواسطة ال Compiler	يدوياً بواسطة المبرمج بـ ()	يدوياً باستخدام كود مخصص	
اتجاه الحجم من صغير لكبير (آمن)	من كبير لصغير (غير آمن)	بين أنواع مختلفة تماماً (كالنص والرقم)	
فقدان البيانات لا يوجد	احتمال كبير (حذف الكسر مثلاً)	لا يوجد (يا التحويل ينجح يا يطلع خطأ)	

BitConverter

تحويل رقم الي " Array of bytes " اللي بينتمي ليها

```
var number = 10;
var bytes = BitConverter.GetBytes(number); // بجيب البايتس اللي بينتمي ليها
الرقم دا

foreach (var b in bytes)
{
    //Console.WriteLine(b);
    //255    -> // array of byte of int of 255  4 byte as int is 4 bytes
    //0
    //0
    //0
    var binary = Convert.ToString(b, 2).PadLeft(8, '0'); // بحول كل بايت الي
    10 و 1 بيتس من
    Console.WriteLine(binary); //1010
                                //0
                                //0
                                //0
}
```

وطبعا في تحويلات تانيه كتير زي من string ل Binary و Hexadecimal وغيره من تحويلات